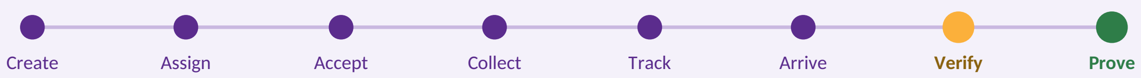
TRACE

Delivery Tracking & Verification Platform

Concept Note & Technical Proposal
Expression of Interest — Software Development Opportunity

CATEGORY	Cloud-Based Delivery Tracking Application
INTEGRATION	SAP Business ByDesign
PLATFORM	Installable Progressive Web Application
TEAM	University Student Development Team — 3 Developers
DATE	August 2026

THE DELIVERY LIFECYCLE



Verified

Geofenced confirmation. A delivery cannot be closed away from its destination.

Universal

Three-tier confirmation. Every delivery closes, smartphone or not.

Resilient

Offline-first field operation. Riders keep working through dead zones.

Affordable

Costed architecture. Near-zero running cost at pilot volume.

01 Executive Summary

TRACE is a delivery tracking and verification platform built on one principle: a delivery is not complete when the rider says so — it is complete when the system can **prove** it was received.

The platform delivers the three interfaces specified in the brief, each designed for a genuinely different operating context rather than one dashboard reskinned three times.

INTERFACE	USER	DELIVERY METHOD
Rider	Delivery riders in the field	Installable mobile application
Customer	Recipients	Installable app, or a secure link requiring no installation
Management	Dispatch and operations	Responsive dashboard — TV, laptop, tablet, phone

Delivery records originate in SAP Business ByDesign and flow into TRACE through a dedicated integration adapter. Confirmations flow back along the same path.

What distinguishes this proposal

- **Geofenced confirmation** — the server rejects a completion attempt made away from the destination. A rider cannot close a job from home.
- **Three-tier confirmation** — every delivery closes regardless of whether the recipient owns a smartphone.
- **Offline-first field operation** — riders continue working through dead zones; the action queue syncs on reconnection.
- **Costed architecture** — we state what the system costs to run each month, and the engineering decisions that keep it there.

02 The Problem

Delivery operations fragment across phone calls, messaging applications and paper records. The specific failures are consistent across organisations:

- Management cannot see where riders are without calling them.
- Customers call the business repeatedly to ask where their package is.
- Delivery notes exist in SAP but never reach the rider's hand.
- Nothing proves a package was actually received.
- Delays are discovered after the fact, not while they can still be corrected.

The opportunity is not another consumer delivery marketplace. It is the operational infrastructure that makes an organisation's existing delivery work visible, verifiable and accountable.

Create → Assign → Accept → Collect → Track → Arrive → Verify → Complete

03 Why a Progressive Web Application

TRACE is built as an installable Progressive Web Application. This is a deliberate engineering decision, not a reduction in scope, and it is the correct one for this deployment.

DECISION DRIVER	RATIONALE
Instant updates	Field software changes constantly in its first months. A PWA updates the moment we deploy. App-store review cycles would leave every bug fix days behind the problem it solves.
Zero install friction	The brief requires recipients to confirm receipt, including those without the app. A recipient who taps *Received* four seconds after opening a link will always outperform one asked to download 40MB first.
One codebase	A three-person team cannot maintain separate Android, iOS and web builds in any realistic timeline. One codebase means every hour of work improves all three interfaces.
Low data footprint	Under 5MB initial load, cached thereafter. This matters at Ugandan mobile data prices, for riders paying their own bundles.

> **Phase 2 — Play Store presence.** The application will be wrapped as a Trusted Web Activity and published to the Google Play Store, providing a native install experience from the same codebase. This is approximately one day of work and requires no rewrite.

04 System Architecture

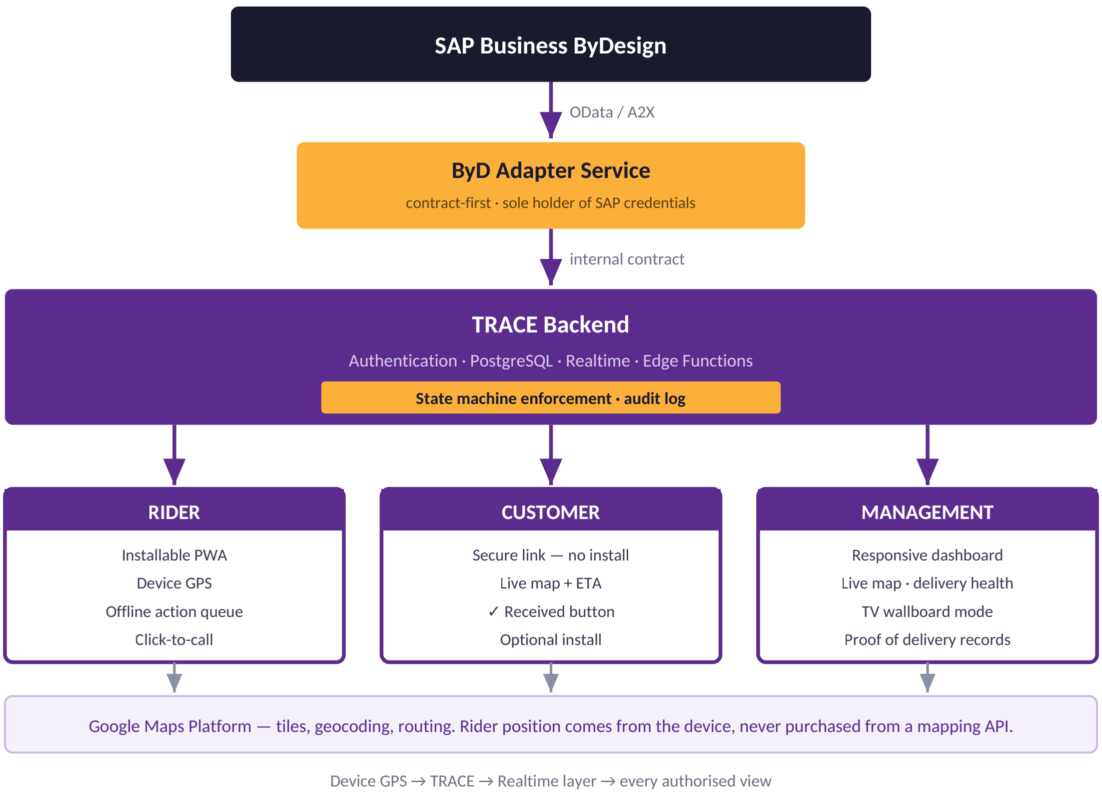


Figure 1 — System architecture. The adapter is the only module aware of SAP data structures.

Two architectural commitments

The adapter isolates SAP entirely. TRACE maintains its own delivery model; the adapter translates between the two. Migrating to a different enterprise system later means rewriting one module, not the platform.

Rider position originates on the device. Google Maps supplies map tiles, geocoding and routing — it is never asked where the rider is. This keeps the tracking system under our control and API costs predictable.

05 User Flows

Each interface is designed for a different operating context. The rider is standing beside a running motorcycle and has three seconds of attention. The recipient wants one screen and one button. The dispatcher is reading a wall from three metres away.

5.1 Management — Create and Dispatch

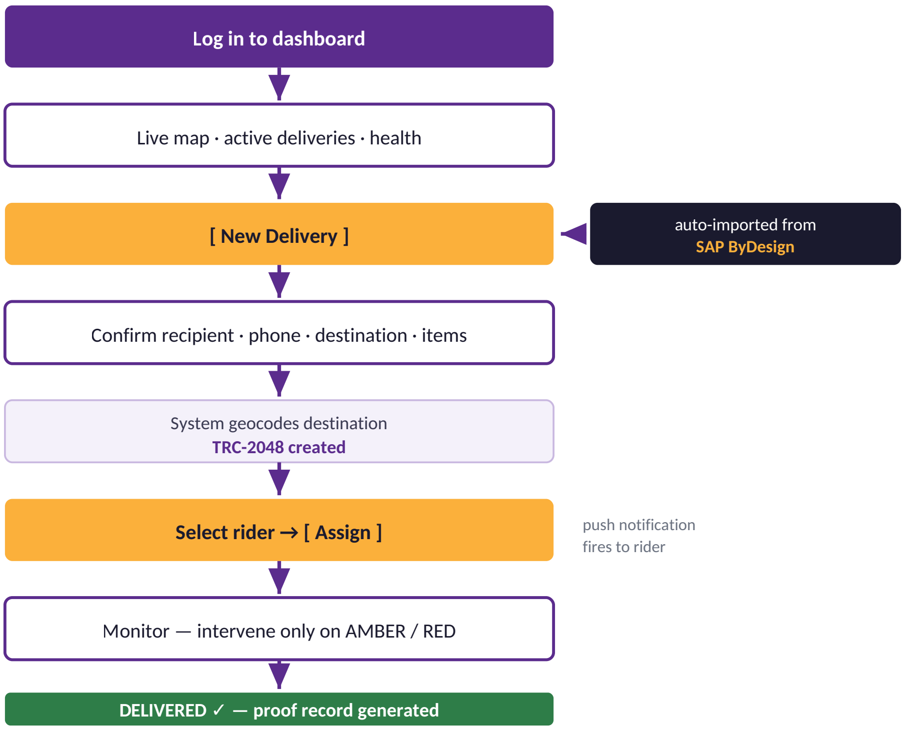


Figure 2 — Management flow. Deliveries enter manually or automatically from SAP.

5.2 Rider — Field Operation

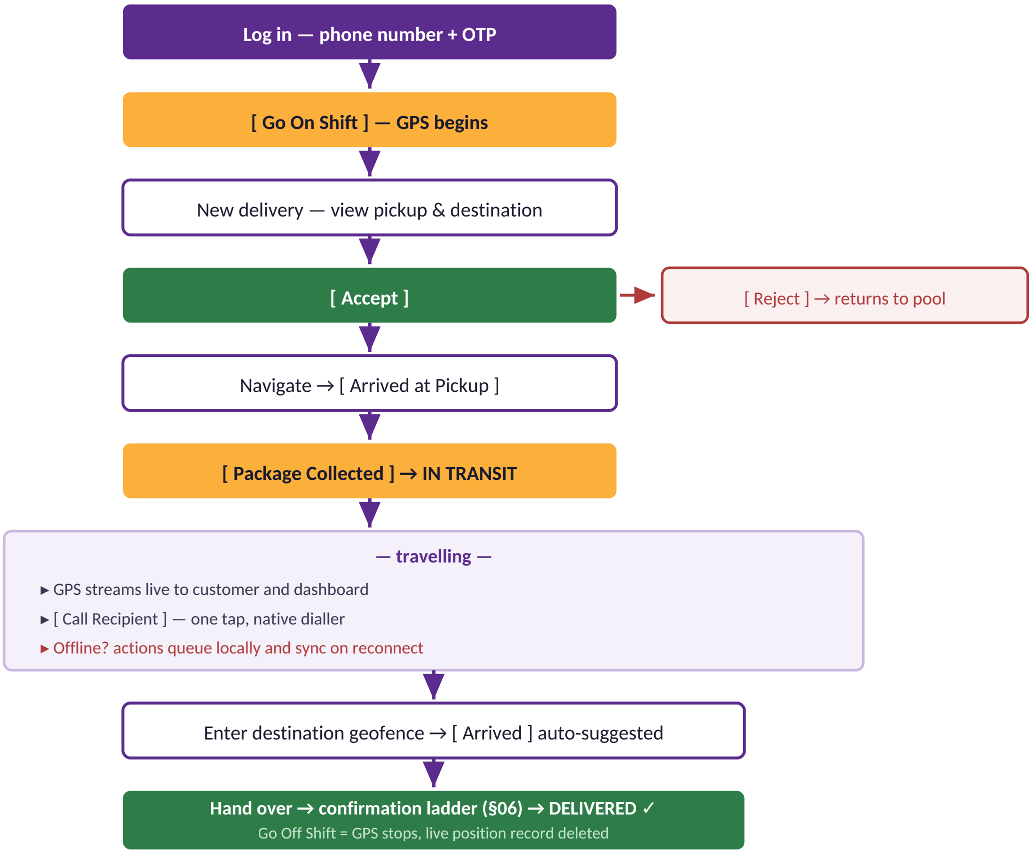


Figure 3 — Rider flow, including the offline branch and the shift-bound tracking boundary.

> **Interface constraints for the rider.** One primary action per screen. Thumb-sized targets. Maximum contrast for direct sunlight. Dark theme by default — night deliveries are common, and it conserves battery on OLED screens. A rider should never need to read a form.

5.3 Customer — Track and Confirm

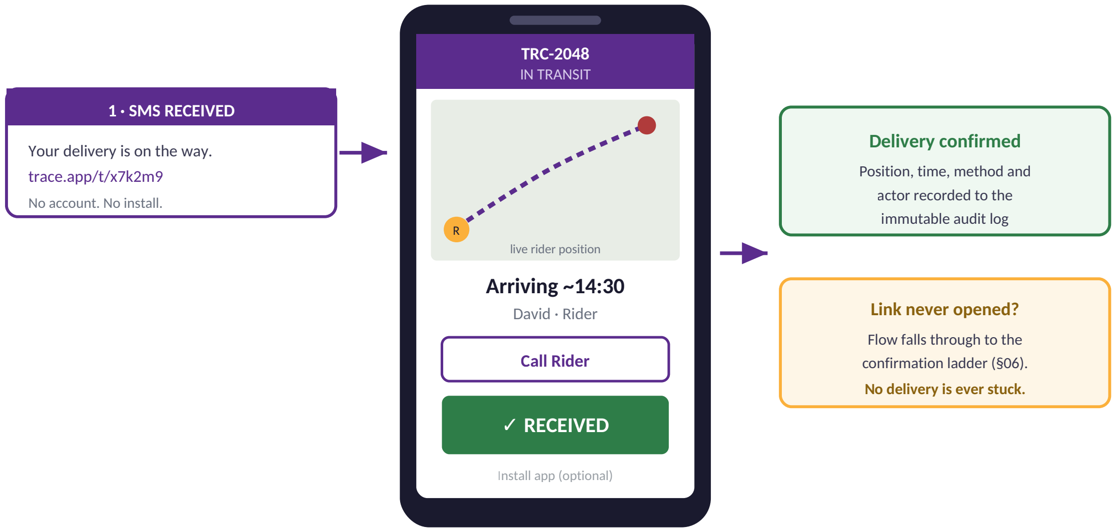


Figure 4 — Customer flow. One link, one screen, one button.

5.4 Operations Wallboard

Opened full-screen on a television in the dispatch office. It requires no interaction — it is designed to be read at three metres.

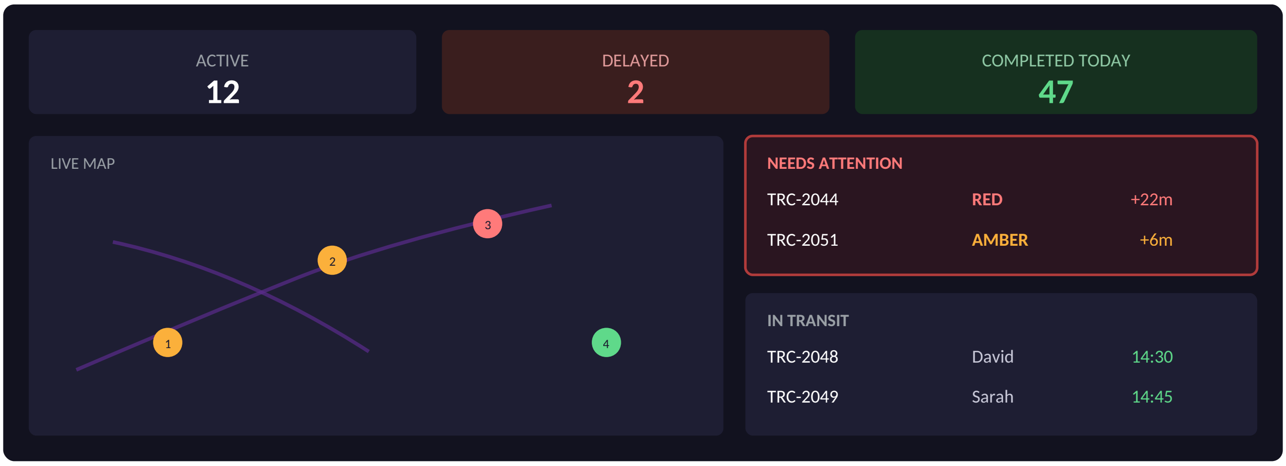


Figure 5 — Operations wallboard. Exception-based, high-contrast, zero interaction.

> **Delivery Health.** Every active delivery is classified GREEN, AMBER or RED against its expected arrival time. The dashboard's purpose is not to show management the map — it is to tell them where attention is required.

06 Confirmation — Three Tiers

The brief requires a *"Received"* button, in the customer's app or on the rider's device. TRACE implements exactly that, with two fallbacks beneath it so that no delivery can be left open.

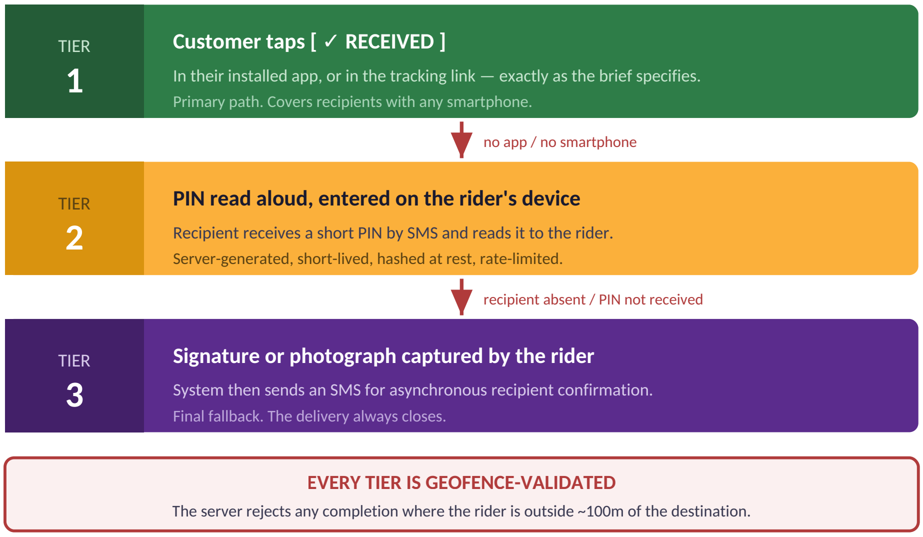


Figure 6 — Confirmation ladder. The requirement is met literally at Tier 1 and exceeded beneath it.

Confirmation is not a button that sets a flag. It is a server-validated state transition that records position, timestamp, method and actor to an immutable log.

07 Delivery State Machine

Status is server-controlled. Clients request transitions; they never write status directly.

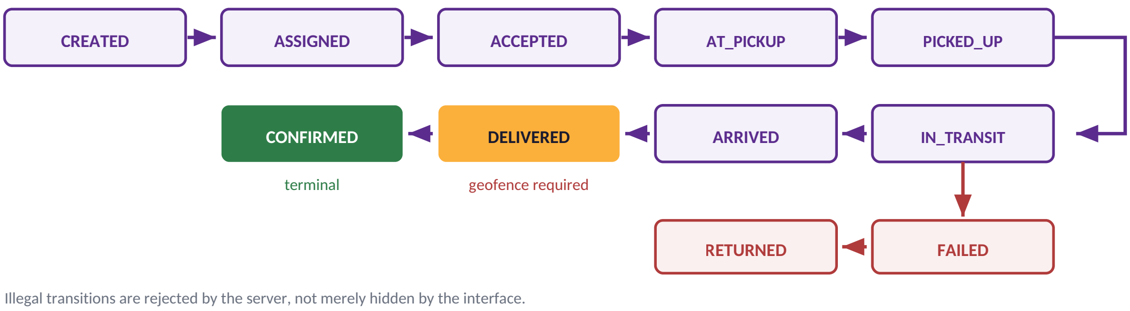


Figure 7 — Delivery state machine. Handover (DELIVERED) and verification (CONFIRMED) are deliberately separate events.

Every transition writes an **append-only event** recording actor, position, device time and server time. Nothing is ever updated or deleted. The gap between device time and server time is the evidence that an offline synchronisation is legitimate — and it is what settles a disputed delivery.

08 SAP Business ByDesign Integration

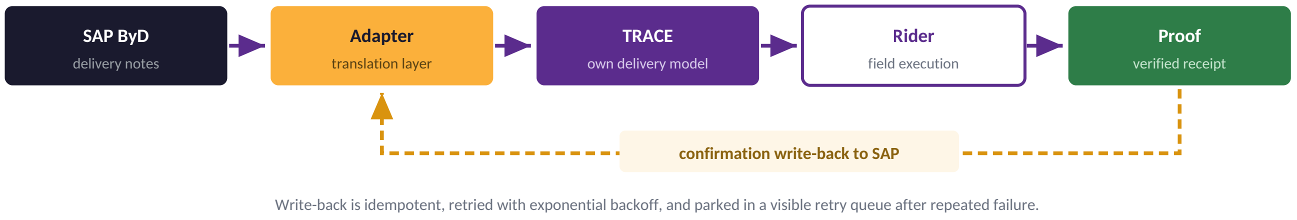


Figure 8 — Integration path. A confirmation is never silently dropped.

The adapter exposes a fixed interface — fetch delivery notes, push confirmations, health check — with two implementations behind it: a mock backed by seeded fixtures, and a live implementation bound to a real tenant. A single environment variable switches between them; nothing above the adapter changes.

> **We will not claim a live SAP connection we do not have.** Exact service names and authentication mode vary by tenant configuration and scoping, and would be confirmed in a short session with the ByDesign administrator before the live adapter is written. The prototype demonstrates the complete integration path against the mock — which proves the architecture without misrepresenting it.

09 Security

CONTROL	IMPLEMENTATION
Identity	Phone number with OTP. Roles held as server-side claims, never as a client-editable field.
Status integrity	Clients cannot write *status*. Only server-validated transitions change delivery state.
Row-level security	Riders see assigned deliveries; customers see their own; management is scoped to its organisation.
Anti-fraud	Geofenced completion. Rider position is validated server-side before a delivery can close.
Tracking links	Long, random, non-sequential tokens that expire once the delivery completes.
OTP handling	Server-generated, short-lived, hashed at rest, rate-limited. Plaintext is never stored.
Audit	Append-only event log. No deletions, no edits, full forensic trail per delivery.
Secrets	SAP and server credentials in managed secret storage. Browser map keys domain-restricted and scoped.

> **Location privacy by design.** GPS runs only during an active shift. Going off shift deletes the live position record entirely. Riders are not tracked when they are not working — enforced structurally rather than by policy. This aligns with Uganda's Data Protection and Privacy Act, 2019.

10 Offline Resilience

Connectivity in the field is intermittent. The rider application maintains a local action queue that preserves work through dead zones.

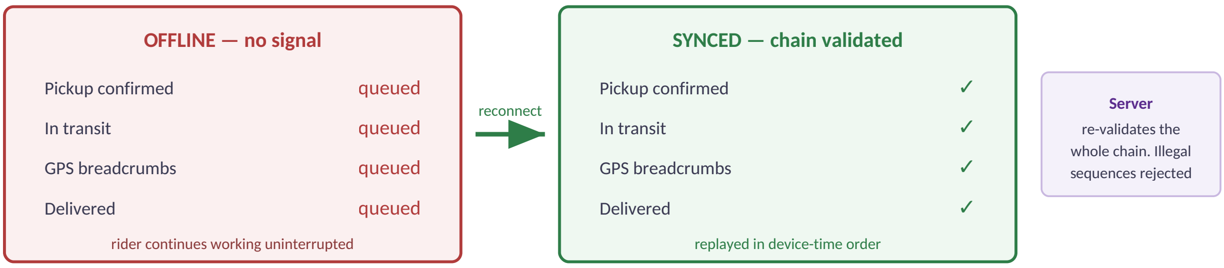


Figure 9 — Offline action queue. This will be demonstrated live, in airplane mode.

11 Technology Stack

LAYER	TECHNOLOGY	RATIONALE
Frontend	Next.js + TypeScript	One codebase serving all three interfaces
Styling	Tailwind CSS	Shared design tokens across app and dashboard
Application model	Progressive Web App	See §03
Backend	Supabase	Authentication, database, realtime and functions in one platform
Database	PostgreSQL	Relational integrity for an audit-critical system
Realtime	Supabase Realtime	Live position without custom socket infrastructure
Local storage	IndexedDB	Offline action queue
Mapping	Google Maps Platform	Tiles, geocoding, routing
Integration	Custom ByD adapter	See §08
Design	Figma	Wireframes before code

What we deliberately do not build: a database engine, an authentication platform, a realtime layer, a map engine or a routing engine. Proven services carry the infrastructure so that our effort goes into delivery operations, verification, integration and usability.

12 Running Cost

An operational system must be affordable to run, not only to build. Cost control is an architectural concern here, not an afterthought.

LEVER	EFFECT
Adaptive GPS ping rate	Sparse in transit, frequent near the destination — reduces database writes and rider data consumption
Batched position history	Breadcrumbs stored in 60-second segments rather than one record per fix
Cached geocoding	Repeat destinations resolve from cache and are never re-billed
Throttled route recalculation	Recalculated on significant deviation, not on a timer
Device-sourced GPS	Rider position is never purchased from a mapping API

At pilot volumes the platform operates within the free tiers of the chosen services. Monthly projections at 100, 1,000 and 5,000 deliveries — verified against current published pricing — accompany the prototype submission.

> **No paid AI services are used.** They are not required by the brief, and they would introduce recurring cost to a system whose value rests on reliability and verifiability.

13 Delivery Timeline

Development window: **18 – 30 August 2026**. Twelve working days with a two-day contingency at the end.

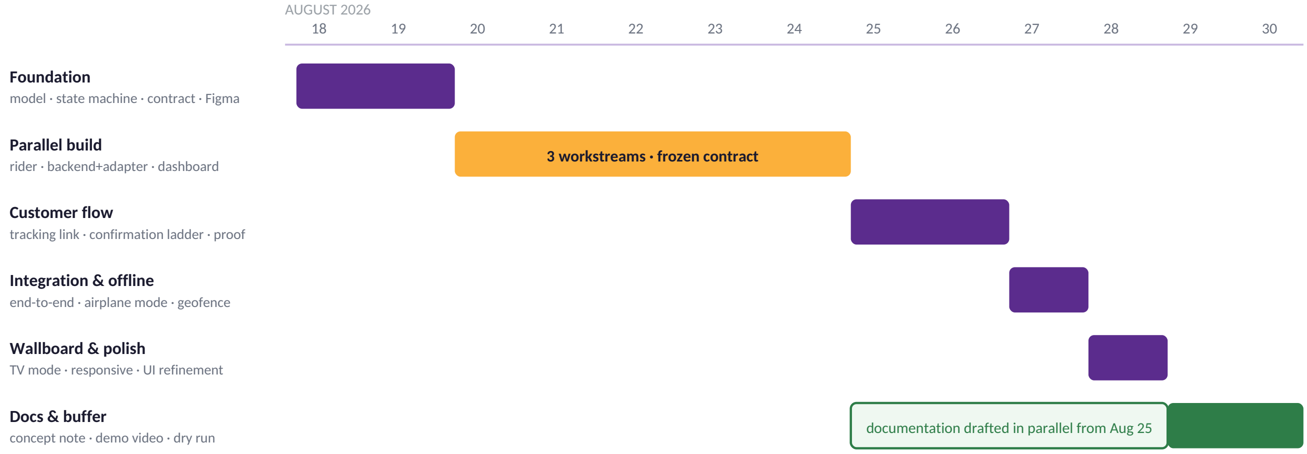


Figure 10 — Delivery schedule. Documentation runs alongside development rather than after it.

> **Post-award phasing.** Should TRACE be selected, live SAP tenant binding, Play Store publication and advanced analytics are scoped as Phase 2 following MVP handover.

14 Team

Three developers, three workstreams, one shared repository, and a data contract frozen on day one.

DEVELOPER	OWNERSHIP
1 — Backend & Integration	Database, authentication, state machine enforcement, security rules, ByDesign adapter
2 — Rider Experience	Rider PWA, GPS, offline queue, delivery workflow, notifications
3 — Customer & Management	Tracking link, confirmation flow, dashboard, TV wallboard, design system

All three interfaces are built against the same design tokens, so the product reads as one system rather than three separate applications.

15 Scope Boundary

Discipline about what is *excluded* is what makes the timeline credible.

IN SCOPE FOR THE MVP

Three interfaces · SAP adapter with a demonstrable integration path · live GPS tracking · ETA · click-to-call · three-tier confirmation · geofenced completion · proof of delivery · offline action queue · management dashboard · TV wallboard · full security model.

EXPLICITLY OUT OF SCOPE

Multi-stop routing · automated dispatch optimisation · payments and wallets · marketplace features · loyalty and pricing systems · AI features · advanced analytics · multi-organisation tenancy.

Each excluded item is a defensible Phase 2 addition. None is required to prove that the platform works.

16 Demonstration Scenario

One delivery, end to end, in under four minutes.

STEP	ACTION
1	Delivery note arrives through the SAP adapter — **TRC-2048**, documents to Ntinda
2	Management assigns rider **David**; push notification fires
3	David accepts and confirms pickup
4	Recipient receives SMS and opens the tracking link — **no installation**
5	Live map shows David moving; ETA updates dynamically
6	Dashboard and TV wallboard reflect the same delivery in real time
7	**Airplane mode** — David confirms arrival offline; the action queues locally
8	Connection returns; the queue synchronises; the server validates the full chain
9	Recipient taps ✓ **Received**; the geofence check passes
10	**DELIVERED** — proof record generated with position, time, method and actor
11	Confirmation writes back through the adapter to SAP
12	Wallboard updates without a refresh

Steps 7 and 8 are the ones the panel will remember.

17 Risk Management

RISK	MITIGATION
Poor connectivity in the field	Offline action queue with server-side chain validation on reconnection
No live SAP tenant during the build	Contract-first adapter, mock-backed, honestly declared in this document
Mapping API cost overrun	Device-sourced GPS, cached geocoding, throttled route recalculation
GPS inaccuracy	Accuracy recorded with every fix; geofence radius tuned against it
Unauthorised tracking access	Expiring non-sequential tokens and row-level security
Scope expansion	Explicit exclusion list and a data contract frozen on day one
Timeline slip	Two-day buffer, parallel documentation, post-award work deferred to Phase 2

18 Alignment with Evaluation Criteria

WEIGHT	CRITERION	HOW TRACE ADDRESSES IT
40%	**Technical feasibility & architecture**	Modular separation of frontend, backend, integration and mapping. A server-enforced state machine with an append-only audit trail. A contract-first SAP adapter that is explicit about what has and has not been tested against a live tenant. Running cost modelled as an engineering constraint.
30%	**UI/UX quality**	Three interfaces designed for three genuinely different contexts — a rider beside a running motorcycle, a recipient who wants one screen and one button, a dispatcher reading a wall from three metres. Not one dashboard reskinned three times.
20%	**Innovation & value addition**	Geofenced completion that makes falsified delivery impossible. Three-tier confirmation that closes every delivery regardless of recipient device. Offline-first operation designed for actual field conditions. Exception-based management that tells operators where to look.
10%	**Realism of delivery timeline**	A dated schedule with named outputs, a two-day contingency, an explicit exclusion list, and post-award work honestly deferred to Phase 2.

19 Conclusion

TRACE does not attempt to be a marketplace. It is the operational layer that makes an organisation's existing delivery work visible, verifiable and accountable.

The architecture is modular. The interfaces are simple. The tracking is real time. The confirmation is server-validated and geofenced. The integration is honestly scoped. The system runs on infrastructure that costs almost nothing at pilot volume.

Most importantly, it is achievable by three developers in twelve days — and every component built is production-grade, not demonstration scaffolding.

Create it. Track it. Verify it. Prove it.